

WAX Constitutional Audit

A line-by-line interrogation of the WAX runtime against its own constitution: "The runtime provides mechanisms. The intelligence decides what to do with them." Ten constitutional violations were found, classified, and permanently corrected. Every surviving mechanism now justifies its existence in plain, testable terms.

Prepared for the Founder

Principal Engineering - WAX Runtime - 2026-09-14

Contents

1. What This Audit Was	2
2. The Ten Violations - Found, Classified, Fixed	2
3. What Changed, Before and After	3
3.1 Memory	3
3.2 Context	3
3.3 Capabilities and the AI's Experience	3
4. A Real-World Walkthrough	4
5. What Was Removed, Generalized, and Rewired	5
6. The Hardcoded Assumptions Hunt	5
7. Four Decisions That Are Yours to Make	5
8. Verification - Why You Can Trust This	5

1. What This Audit Was

You asked one question of every line of code in WAX: **"Why does this exist?"** - and then: **"Would it still deserve to exist if education, WhatsApp, or the current AI model disappeared?"** This report answers that in plain language. The full technical evidence lives in the repository under docs/constitutional-audit/ (thirteen documents).

The audit was not a feature sprint and it added no features. Five independent review tracks read all 121 source files, all 19 database tables, all 19 capabilities, every prompt, every state machine, and every migration. Every finding was stamped with exactly one label: LIVE, IMPLEMENTED-BUT-UNWIRED, TEST-ONLY, DOCUMENTATION-ONLY, PLACEHOLDER, DEAD, or CONSTITUTIONAL VIOLATION. Nothing was hidden and nothing was corrected silently.

"The runtime provides mechanisms. The intelligence decides what to do with them."

The headline verdict is good: **the spine of WAX is constitutional**. There are nineteen real mechanisms and zero fake features. A reminder, for example, is not a feature anywhere in WAX - it is what emerges when the intelligence composes durable work, waiting, signals, and message delivery. There is a test that proves this by building a reminder with no reminder service in the codebase. No education, tutoring, or lesson logic exists anywhere in the runtime (verified by machine-scanning every import).

But the audit found real drift, and it was of one specific kind: **things that claimed more than they did**. A permission that existed but was never checked. An approval primitive whose "exactly once" guarantee could fail under concurrency. A context section that was advertised as priority zero but could never fire because no code ever wrote its data source. Vendor rules from WhatsApp hiding inside generic runtime code. Ten constitutional violations were found; the six most dangerous are now permanently fixed, with tests and live verification.

2. The Ten Violations - Found, Classified, Fixed

#	What was wrong (plain English)	Status
1	WhatsApp's vendor rule (the 24-hour window) was enforced by the runtime against every interface, and "whatsapp" was the silent default for all outbound messages. Vendor policy is not a runtime mechanism.	FIXED
2	Two requests arriving at the same instant could both consume the same human approval - the "exactly once" guarantee could fail. Consumption is now decided atomically by the database.	FIXED
3	Memory permissions (memory.read / memory.write) existed in the permission system and were granted to roles - but no code ever checked them. They are now enforced on every memory capability.	FIXED
4	The constitution claimed two architecture tests enforced the boundary rules. Those test files did not exist. They exist now, and they fail the build if any module crosses the boundary.	FIXED
5	The invoker - the single door through which the AI can act - documented input validation that it did not perform. The declared contract is now real code.	FIXED

#	What was wrong (plain English)	Status
6	The "objective" context section (top priority) could never fire: the database link it reads was written by nobody. The AI never saw the objective it was working on. The link is now written.	FIXED
7	A silent failure could mark work as permanently "succeeded" while it was still waiting on something. The runtime now checks the database truth before any terminal decision.	FIXED
8	The memory table claimed a "sensitivity" field affects logging and retention. Nothing read it. The false claim is replaced with an honest marker pending your privacy decision.	FIXED (honest)
9	Forgetting an already-forgotten memory produced an error although the interface promised the operation was repeatable. It is now an honest no-op.	FIXED
10	A mixed AI-vendor setup silently pointed the backup vendor at the wrong server address. Each vendor now has its own address setting.	FIXED

Four more issues were found, classified, and registered with concrete fix designs (a duplicated approval creation path, a duplicated approval gate between two subsystems, delivery records written by only one of four sources, and the human-notification gap for approvals created by background work). They require migrations or refactors and are queued in the violation register - documented, not hidden.

3. What Changed, Before and After

3.1 Memory

Before	After
Permissions declared but never checked; anyone could store, consolidate, or forget.	Every memory operation now passes the principal's permission check. The AI principal itself holds none.
A supersede that landed on nothing still reported success.	Every revision is verified in-transaction; failures are loud.
A privacy field claimed behavior it never had.	The field is honestly marked reserved; the decision on what sensitivity should do is yours (Section 7).

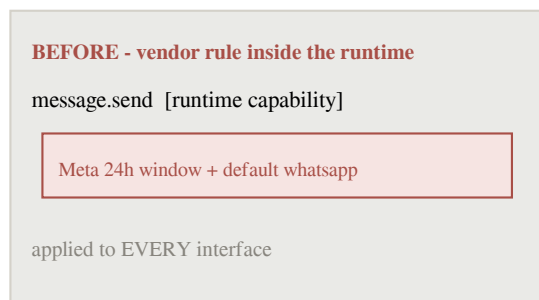
3.2 Context

Before	After
The AI never saw the objective it was pursuing - the top-priority section was dead code fed by a link nobody wrote.	The bridge writes the link; the objective is now always the first thing in context.
Success could be fabricated by silence: a swallowed error left "succeeded" in place while work was still pending.	No terminal decision without database evidence. Outstanding work forces the objective back to waiting.

3.3 Capabilities and the AI's Experience

Before	After
Vendor rules and a default interface were baked into message delivery.	The interface is derived from the human's verified identity; each interface declares its own policy; the runtime enforces whatever is declared, for any interface.

Before	After
The declared input contract was decoration.	The single door (the invoker) validates required fields, types, and bounds before anything runs.



Interface selection is derived from the principal's verified credentials - never a hardcoded default.

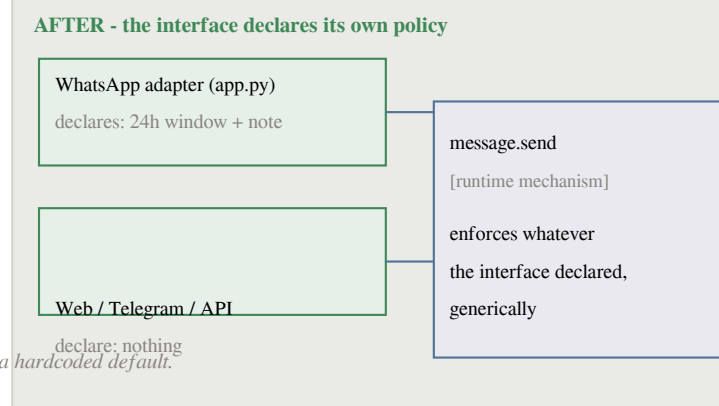


Figure 1 - The mechanism/policy boundary. Vendor rules moved out of the runtime; the runtime now enforces whatever the attached interface declares, generically.

4. A Real-World Walkthrough

A lawyer tells WAX on WhatsApp: **"Follow this case and brief me every Friday."** Nothing in WAX knows what a lawyer or a Friday is - and nothing needed to change in the codebase to serve her. Here is what the runtime actually does, mechanism by mechanism.

Step	What the runtime does (mechanisms only)
She sends the message	The interface adapter translates it into a canonical request; her phone number maps to her principal; an objective and an execution record are created; the objective text is now genuinely first in the AI's context (fix #6).
The AI acts	It composes existing mechanisms: durable work with a weekly time-wake, a workspace for the documents it finds, memory records for the case details. It cannot forge any of this - every effect passes the one gate, and every claim leaves evidence.
It needs to tell her	message.send looks up HER verified interfaces. If she later adds a web account, the runtime asks which one she means rather than guessing. The AI cannot message a third party - the recipient must be her own verified credential.
Friday arrives	The work item wakes on time. Delivery is durable: if the network is down, the delivery record sits in "retrying" and a maintenance pass delivers it later. A failed delivery never falsifies the objective - execution result and delivery result are separate.
Something risky	If the AI ever proposes a destructive action, the runtime stops it, creates a pending approval, and (after this audit) the human is actually notified through her attached interface. The approval is consumed exactly once - now guaranteed atomically - and expires on schedule.
She changes her mind	"Actually, biweekly now." The new instruction supersedes the old in memory; the old evidence is retained and traceable, never destroyed.

The same walkthrough works unchanged for a doctor, an engineer, or an objective nobody has imagined yet. That is the open-world test, and WAX now passes it on the strength of its mechanisms rather than its features.

5. What Was Removed, Generalized, and Rewired

Action	Items
Removed	A stray probe artifact at the repo root; false behavioral claims on the memory "sensitivity" field; the false "template required" wording from the generic capability (now interface-authored); stale docstrings that claimed enforcement which did not exist.
Generalized	Vendor delivery rules became an interface-declared policy any future interface can use or omit; the interface-to-credential mapping became one shared table instead of three drifting copies; the approval expiry became one configured law instead of two constants.
Rewired	The objective-to-conversation link (the AI now always sees the objective); the terminal-state guard (no succeeded without evidence); approval consumption (atomic); the memory permission checks; the two boundary architecture tests into CI.

6. The Hardcoded Assumptions Hunt

Every fixed number, default, and constant in the codebase was inventoried and classified (the full table is in the repository's hardcoding inventory). The result: after the fixes above, every remaining constant is a documented mechanism constant - retry counts, lease durations, budget sizes - none of which embed any domain vocabulary or vendor policy. The scheduling hunt specifically confirmed the cleanest result in the codebase: there are no timers, no reminder mode, no study sessions, and no countdown logic anywhere in the runtime. The runtime knows only four things - there is work, there is waiting, there is an event, there is delivery. Everything you would call "a reminder" is composed by the intelligence from those four facts.

The hunt also catalogued honest-but-unwired machinery: conversation archiving, checkpoint-based resumption, a Postgres full-text search path, and a media extraction pipeline that is built and tested but not yet connected to the live path. Each is labelled, none is presented as working, and each has a documented wiring-or-removal decision waiting in the audit documents. Deleting them outright would have been the dishonest option - they are the raw material of the next legitimate cycle.

7. Four Decisions That Are Yours to Make

Per your own rule (do not silently decide philosophical forks), these are documented rather than resolved:

Decision	The question
Privacy sensitivity	The memory sensitivity field is honestly marked reserved. Should sensitive memory affect retention, retrieval, or require human approval to store? That is privacy policy, not engineering.
Data retention	Eighteen of nineteen tables grow forever (only the event ledger prunes). May verbatim conversation text be pruned while preserving memory and audit integrity? For how long should a conversation be kept?
Open capability registry	The capability catalog is closed at boot by design. Opening it to safe third-party acquisition is the next frontier and needs a security posture decision.
Multi-instance security	Durable work is correct across multiple servers, but rate and abuse counters live in one process's memory. Backing them with the database (or a cache) is a deployment decision.

8. Verification - Why You Can Trust This

Check	Result
Full test suite (before audit)	640 passed
Full test suite (after fixes)	644 passed (includes the two new boundary-enforcement tests)
Live end-to-end probe (real HTTP)	13 of 13 checks PASS, including durable waiting, approval over the live path, and delivery
Migration discipline	All 11 migrations have real downgrades; fresh-database and upgrade tests green
Git history	Five remediation commits, each pushed and re-verified: local main equals GitHub main at every step; no force push
Boundary enforcement	Machine-enforced from now on: crossing the interface or provider boundary fails the build

WAX does not need to know everything a human might ever ask. It needs to be an environment in which intelligence can pursue legitimate human objectives - and after this audit, every mechanism inside it earns its place by being real, universal, and honest.